

Prof. Oscar Víquez Acuña.

Compilador  
Mini GO

## **Introducción**

Un compilador/intérprete es una herramienta de trabajo muy importante para cualquier estudiante de computación y normalmente dichas herramientas se utilizan sin conocer detalles de su implementación que podrían ser de ayuda para comprender mejor el funcionamiento de la herramienta. Se espera que después de este proyecto, el estudiante tenga con una mejor comprensión de los compiladores/intérpretes que utiliza y al mismo tiempo incremente sus habilidades de programación OO.

**Cualquier detalle extra que no sea aclarado en esta documentación y que surja debido a las dudas de los grupos, será aclarado en su momento por el profesor sin necesidad de elaborar otro enunciado.**

## **Definición**

El proyecto general consiste en implementar un compilador para un lenguaje subconjunto de GoLang que llamaremos “Mini GO”. El objetivo es realizar las fases de análisis sintáctico, chackeo de tipos y generación de código para el compilador del lenguaje, siguiendo los patrones y herramientas desarrollados/utilizadas en clase. Para el desarrollo de esta etapa será necesario la utilización de la herramientas ANTLR4.

La implementación del proyecto debe realizarse en el lenguaje de programación de su preferencia siempre que no sea Java y que sea compatible con ANTLR4.

Este documento está complementado con el documento “*MiniGo Grammar.docx*”, en donde se muestra la gramática del lenguaje a implementar.

## **Requerimientos Generales**

Se espera que el estudiante sea capaz de implementar la siguiente lista de requerimientos del programa:

- Reconocer por medio del scanner todos los tokens válidos para Mini GO con sus respectivos formatos según el lenguaje original GoLang. Además deben permitirse el manejo de comentarios en el código fuente siguiendo el mismo formato.
- Reconocer tipos de datos válidos según las reglas de tipo establecidas más adelante
- Generar código de máquina real para arquitectura x86 utilizando LLVM.
- Escribir archivos de prueba que ejemplifiquen todas las capacidades del lenguaje.
- Implementar un Editor de texto que permita mostrar y compilar las pruebas elaboradas. El editor debe mostrar además los posibles errores, el lugar en donde estos eventualmente

aparezcan y debe permitir identificar fácilmente por línea y columna estos errores en el editor. El editor puede ser implementado o descargado/adaptado a partir de cualquier librería/framework disponible en el lenguaje seleccionado para la implementación u otro.

## **Reglas de tipos para el lenguaje**

Los siguientes son los elementos semánticos presentes en Mini Go que se DEBEN cumplir siguiendo las mismas condiciones que en Golang. Puede que no estén presentes todos los elementos semánticos del lenguaje o no se aclaren al máximo, por lo que será clave el entendimiento del funcionamiento del lenguaje padre GO para realizar todas las validaciones posibles :

1. Existen tipos array y tipos slice que tiene sus particularidades que deben considerarse. Para los tipos slice, deben considerarse las funciones preestablecidas append, len y cap (para efectos de tipos de entrada y de retorno)
2. Existirán además estructuras tipo registros con el mismo formato de Golang. Tanto la definición de la estructura como su posterior uso deben ser validados y verificados.
3. Los identificadores pueden ser globales y locales. Identificadores en diferentes ámbitos pueden tener el mismo nombre.
4. Ningún identificador debe ser declarados dos veces en el mismo ámbito con el mismo nombre.
5. Ningún identificador podrá ser utilizado sin haber sido declarado previamente.
6. Mini Go permitirá hacer inferencia de tipos solamente para tipos primitivos a partir del tipo de dato de las expresiones asignadas. Esto para las declaraciones sin tipo de dato y su tipo de datos será definido a partir del tipo de la expresión única al momento de la declaración.
7. Debe realizarse el chequeo de tipos en las expresiones y algunos statements que así lo requieran.
8. El número y tipo de los parámetros en procedimientos/funciones deben ser igual a los declarados por dicho procedimiento/función.
9. No debe permitir retornar un valor a menos que se encuentre dentro de una función.
10. La expresión de retorno de una función debe o ser inferido o tener el mismo tipo con el que fue declarada la función.

## **Elementos a implementar para generación de código**

No será necesario generar código para la totalidad de las características y variantes del lenguaje. La siguiente es la lista de elementos a implementar:

- 1) Declaraciones y usos de variables de tipos simples tanto globales como locales en asignaciones y expresiones. Solo las dos formas más simples de declaración tipada.
- 2) Declaraciones y usos de arreglos de enteros. La forma más simple de declaración tipada.
- 3) Declaraciones y usos de métodos (procedimientos y funciones)
- 4) Instrucciones de control de flujo solamente IFs y LOOPS, sin uso de “break” ni “continue”.
- 5) Implementación de la función predefinida “println” para imprimir cualquier tipo simple y la función “len” para obtener el largo de un arreglo de enteros.
- 6) Expresiones simples o complejas, con o sin agrupación usando paréntesis para operaciones con los tipos de datos mencionados utilizando los operadores matemáticos tales como +, -, \*, /, % y operadores lógicos tales como ==, >, <, >=, <=, !=, según sea el caso en la expresión.

- 7) Se permitirá el uso de cadenas RawString solamente y para impresiones dentro del “println”

## Documentación

La documentación deberá incluir las siguientes partes y no debe exceder las 10 hojas por un lado sin incluir la portada siguiendo el formato establecido por la carrera para este fin. El contenido debe dar énfasis en lo siguiente:

- Portada formal.
- Análisis del lenguaje (análisis de la gramática, del código que puedo crear o no puedo crear con ella, además de eventuales comparaciones con el lenguaje original)
- Soluciones e implementación. (cuestiones técnica de la implementación del proyecto)
- Resultados obtenidos. (cuadro **explicativo** de lo realizado y lo no realizado – énfasis en esto último)
- Conclusiones **del trabajo**.
- Bibliografía. (de ser necesaria y solo documentos formales con autor)

## Aspectos Administrativos

- La tarea se desarrollará en grupos de máximo dos personas.
- La fecha de entrega será:
  - Grupo 50: Domingo 7 de Junio de 2026 antes de las 10:00 pm.
  - Grupo 51: Lunes 8 de Junio de 2026 antes de las 10:00 pm.
- El medio de entrega será el TEC-DIGITAL solamente.
- Cualquier intento de plagio, copias totales o parciales de otras personas o de Internet, serán castigados con nota de 0. *Una ayuda sincera y comprometida denota compañerismo y profesionalismo, pero una duplicación de código sin más, eso es una copia y no aporta absolutamente nada constructivo en su formación profesional!*